ASCII Dungeon Generator

Project Reference Document

1. Project purpose

This project is an educational exercise in learning JavaScript through the development of a basic ASCII roguelike dungeon generator.

The longer-term objective is to develop a playable, turn-based dungeon environment with procedurally generated levels, a player character, movement, collision detection, and eventually additional roguelike mechanics.

The project is also intended to develop a practical understanding of JavaScript programming. Understanding the code is as important as getting it to work.

The project should therefore develop incrementally. New functionality should not be introduced until the existing functionality is understood.

2. Development philosophy

The following principles should guide all development discussions.

Incremental instruction

- Introduce concepts in small, manageable steps.
- Explain the reasoning behind design decisions, not merely the instructions for implementing them.
- Avoid assuming knowledge of JavaScript features that have not yet been introduced.
- Do not advance through several stages of development in a single response.
- Encourage experimentation and testing after meaningful changes.

Understanding before implementation

The objective is not simply to produce working code. The user should understand what the code does, why it is structured as it is, and how its individual components interact.

When appropriate, use pseudocode, conceptual explanations, and small exercises before introducing actual implementation.

Preserve existing code

- Never rename existing variables, functions, objects, or other established identifiers when revising code.
- Preserve established naming conventions.

- When modifying existing code, work from the actual code supplied by the user rather than reconstructing it from memory.
- Avoid unnecessary refactoring or abstraction.

The user sometimes anticipates upcoming implementation steps as a way of testing their understanding. Allow this and evaluate the proposed approach before providing further instructions.

3. Current project architecture

The project uses separate JavaScript modules to distinguish dungeon construction, player behavior, collision detection, and rendering.

File	Responsibility
`index.html`	Basic webpage structure and entry point
`style.css`	Display styling
`main.js`	Application coordination and dungeon display
`dungeon_gen.js`	Procedural dungeon construction
`player.js`	Player initialization and input handling
`collision.js`	Determines whether a proposed destination is occupiable
`renderer.js`	Reserved for future display-related functionality

The current architecture deliberately separates the dungeon's underlying data from its visual representation.

For example, the player character is displayed as `@`, but the underlying dungeon array is not modified to insert that character. The player is rendered as an overlay.

This means that moving away from a staircase reveals the staircase beneath it.

4. Dungeon representation and generation

The dungeon is represented as a two-dimensional JavaScript array: an array of rows, each containing an array of individual map cells.

Current dimensions:

- Height: 60 rows.
- Width: 100 columns.

Current tile representations:

Character	Meaning
`#`	Solid rock
`.`	Excavated floor
`<`	Up staircase
`>`	Down staircase
`@`	Player character (display overlay)

The generator initially fills the entire dungeon with # characters.

Rooms are then excavated by replacing rock with floor tiles.

Room generation

- Room heights are odd numbers between 3 and 9.
- Room widths are odd numbers between 3 and 15.
- Room placement uses center coordinates.
- A five-cell frame is maintained around the map.
- Room overlap is intentional.

The following established variable names must be preserved:

- `center_room_row`
- `center_room_column`
- `top_room_row`
- `bottom_room_row`
- `left_room_column`
- `right_room_column`

These names describe the geometric roles of the values rather than the methods used to calculate them.

Corridor generation

Rooms are connected sequentially using L-shaped corridors.

The current implementation connects the previous room center to the current room center, excavating a horizontal segment followed by a vertical segment.

More sophisticated room-connection algorithms may be considered later, but are not currently necessary.

Staircase generation

The generator collects the coordinates of excavated floor tiles in `floor_coordinates`.

It randomly selects two distinct entries from that collection:

- One becomes the up staircase.
- One becomes the down staircase.

The generator returns an object containing:

- `dungeon`
- `up_stair_pos`

The up staircase position is used to initialize the player.

5. Player initialization and movement

Player initialization is handled by `initialize_player()` in `player.js`.

The player object contains:

- `player_row`
- `player_column`

The player initially appears at the up staircase.

Keyboard input is handled by `handle_player_input()` using a `keydown` event listener and `event.code`.

Current movement scheme:

Numpad key	Direction
8	Up
2	Down
4	Left
6	Right
7	Up-left
9	Up-right
1	Down-left
3	Down-right
5	Wait

The player is rendered by `display_dungeon()` in `main.js`.

The display function iterates through the dungeon array and substitutes `@` whenever the current array coordinates match the player's coordinates.

6. Collision detection

Collision detection is implemented in `collision.js` through the exported function:

```
check_collision(dungeon, proposed_row, proposed_column)
```

The function returns a Boolean:

- `true` means the proposed destination can be occupied.
- `false` means the proposed destination cannot be occupied.

The function checks two things:

1. Whether the proposed row and column fall within the dungeon's boundaries.
2. Whether the destination tile is an occupiable tile.

Currently, `.`, `<`, and `>` are considered occupiable.

The boundary checks must occur before attempting to access the proposed dungeon cell. This prevents invalid array indexing.

Movement and collision sequence

All eight movement directions now follow the same general procedure:

1. Calculate `proposed_row` and `proposed_column`.
2. Pass those coordinates and the dungeon to `check_collision()`.
3. Store the returned Boolean in `valid_destination`.
4. If the destination is valid, assign the proposed coordinates to the player's actual position.
5. Redraw the dungeon.

The distinction between proposed coordinates and actual player coordinates is intentional.

The proposed coordinates describe a possible destination. The collision function evaluates that destination before the player's actual position changes.

Diagonal movement rules

The current implementation checks only the destination tile when moving diagonally.

This permits diagonal corner-cutting when the destination is open, even if one or both intervening cardinal tiles are blocked.

This is a deliberate temporary design decision.

Angband-style and Brogue-style diagonal movement rules have been discussed. No change is currently required.

7. Current development status

The following functionality has been implemented and tested:

- Two-dimensional dungeon array initialization.
- Random room dimensions and placement.
- Room excavation.
- Sequential corridor generation.
- Random staircase placement.
- Player initialization at the up staircase.
- ASCII dungeon display.
- Player character rendering.
- Eight-direction numpad movement.

- Collision detection.
- Dungeon boundary protection.
- Collision checks applied to all eight movement directions.

The initial implementation phase is complete.

8. Future development

The next development phase has not yet been formally established.

Potential future areas include:

- Field of vision and line-of-sight calculations.
- Monster placement and movement.
- Turn-based game logic.
- Combat.
- Additional dungeon features and environmental objects.
- More sophisticated rendering.

These are possibilities, not commitments. Development should proceed one feature at a time.

The existing separation between dungeon construction and display logic should be maintained.

9. Working conventions and version control

The project is maintained independently using Git.

Git is the authoritative record of the source code and its development history.

ChatGPT conversations are used for instruction, design discussion, troubleshooting, and code review. They should not be treated as the authoritative source of the current implementation.

At major development milestones:

1. Commit the working code to Git.
2. Start a new focused ChatGPT conversation when appropriate.
3. Supply the current source files.
4. Use this reference document to establish the project's history, architecture, and design decisions.

The current source files always take precedence over descriptions of earlier implementations in conversation history.

10. Instructions for future ChatGPT conversations

Continue teaching JavaScript incrementally.

Do not assume that because a feature has been implemented, the user necessarily understands every aspect of its implementation.

When introducing new functionality:

- Explain the problem being solved.
- Explain the relevant concepts.
- Discuss the design before presenting substantial code.
- Let the user implement portions themselves whenever practical.
- Review their actual code.
- Correct errors directly and explain their causes.
- Avoid unnecessary changes to working code.

Do not offer praise, compliments, or assessments of the user's input. Keep responses focused on the technical subject.

The central objective remains:

Develop a working roguelike while developing a genuine understanding of the JavaScript used to build it.